

航空部件生命周期与维修管理系统

一、项目背景

在航空器、无人机及其他高可靠装备的运维场景中，部件并不是一次性消耗品，而是具有完整生命周期的工程对象。一个部件从入库、安装、拆卸、维修、重新安装到最终退役，整个过程都必须被准确记录，并且能够在任意时间点追溯其状态、位置、维修历史和使用历史。

本项目要求各小组设计并实现一个以数据库为核心的“航空部件生命周期与维修管理系统”。系统围绕飞机、部件、安装记录、维修记录、飞行记录等对象展开，支持部件安装、更换、维修、退役、查询追溯等功能。项目重点为数据库模式设计、完整性约束、事务处理、历史保留和业务规则落地，具有简单清晰的操作界面。

二、项目目标

理解并设计具有明确业务语义的数据库模式；能够使用主键、外键、唯一约束、检查约束、触发器、事务等机制维护数据一致性；能够处理“历史不可覆盖、过程可追踪”的版本化记录问题；能够设计查询以追溯任一部件在整个生命周期中的安装、维修、使用 and 退役情况。

能够识别 AI 工具生成数据库代码时可能出现的错误，并进行审计与修正。

三、项目任务

完成一个可运行的数据库应用，核心任务包括但不限于以下内容。

系统需要管理飞机信息、部件信息、部件型号信息、安装拆卸记录、维修记录、飞行记录，以及必要的状态与字典信息。系统应支持以下关键业务：

1. 新部件入库，并记录其基本属性、型号、批次、启用状态。
2. 将部件安装到某架飞机上。
3. 将已安装部件拆卸下来，并记录拆卸时间和原因。
4. 对部件进行维修，并记录维修类型、维修时间、维修结果和责任人员。
5. 在不覆盖旧数据的前提下完成部件更换。
6. 记录飞机的飞行任务或飞行日志，并能够将飞行使用情况与安装中的部件关联分析。
7. 对达到寿命上限或损坏不可修复的部件执行退役处理。
8. 查询任一部件的完整生命周期，包括“何时入库、装在哪架飞机上、何时拆卸、维修了几次、是否再次装机、是否已经退役”。
9. 拒绝明显违反业务规则的非法操作。

四、业务规则

项目必须至少实现以下业务规则，且这些规则应尽可能由数据库层保证，而不是只在前端或接口层口头说明。

1. 安装唯一性规则

同一个部件在同一时刻只能安装在一架飞机上，不能同时处于多个有效安装记录中。

2. 历史保留规则

部件更换或拆卸时，不允许直接覆盖旧安装记录，必须保留历史。也就是说，安装记录应体现时间区间或版本信息，而不是只保存“当前在哪”。

3. 维修关联规则

维修记录必须关联真实存在的部件。对于要求“仅已安装部件才允许维修”的小组，可以进一步实现更严格规则；若选择允许“拆卸后送修”，则必须在设计文档中明确说明业务假设，并保证逻辑自洽。

4. 删除限制规则

核心业务数据原则上不允许物理删除。部件退役必须通过状态字段标记，例如 retired，而不是直接 DELETE。若确需删除测试数据，必须与正式业务逻辑隔离。

5. 状态一致性规则

退役部件不能再次安装；已拆卸的安装记录不能被重复关闭；不存在的飞机不能产生安装记录；不存在的部件不能产生维修记录或飞行关联分析结果。

6. 时间合理性规则

安装时间不能晚于拆卸时间；维修结束时间不能早于维修开始时间；飞行时间区间必须合理；部件不应在退役之后仍然出现新的安装或维修活动。

五、推荐数据库模式

各小组可以自主设计，建议至少包含以下实体，可以在此基础上扩展。

1. Aircraft（飞机表）

记录飞机编号、型号、服役状态、启用日期等信息。

2. ComponentModel（部件型号表）

记录部件型号、类别、设计寿命、维修周期、适用机型等信息。

3. Component（部件实例表）

记录具体部件编号、型号、批次、生产日期、当前状态、累计使用时长、是否退役等

信息。

4. InstallationRecord（安装记录表）

记录部件何时安装到哪架飞机上，何时拆卸，安装位置、原因、执行人员等。该表应体现历史，而不是只存当前状态。

5. MaintenanceRecord（维修记录表）

记录某部件的维修工单、维修类型、送修时间、完成时间、维修结论、责任人员等。

6. FlightLog（飞行记录表）

记录飞机的飞行任务、起飞降落时间、飞行时长、任务类型等。

7. ScrapOrRetirementRecord（退役记录表，可选但强烈建议）

记录退役原因、退役时间、审批信息等，以增强可审计性。

8. Operator / Technician（操作人员表，可选）

记录安装、维修、审批等责任主体，有助于体现可追责性。

六、数据库设计最低要求

每个小组必须满足以下最低要求：

至少设计 **6 张核心数据表**；

至少实现 **3 类完整性约束**，如主键/外键/唯一/检查约束；

至少实现 **2 个事务场景**；

至少实现 **1 个软删除或退役标记机制**；

至少实现 **1 个数据库层面的非法操作拒绝机制**，如触发器、约束、存储过程或事务回滚逻辑；

至少实现 **1 类历史保留机制**，禁止通过覆盖旧记录伪造“当前状态”；

至少实现 **3 个具有业务意义的复杂查询**，其中至少 **1 个查询直接体现生命周期追溯**。

七、必须实现的核心功能

功能一：部件入库

录入新部件信息，包含部件编号、型号、批次、状态等。系统必须防止重复部件编号。

功能二：部件安装

将某个可用部件安装到某架飞机的某个位置。系统必须检查：
该部件当前是否已安装在其他飞机上；

该部件是否已经退役；
安装时间是否合理。

功能三：部件拆卸或更换

从飞机上拆下部件，或将旧部件拆下并装上新部件。实现时必须保留旧安装记录，不允许直接 UPDATE 覆盖为“新飞机”或“新位置”。

功能四：维修登记

为部件创建维修记录，并记录维修类型与结果。若你们采用严格模式，可限制为仅对满足特定状态的部件允许维修；若采用宽松模式，也必须在文档中解释维修流程假设。

功能五：部件退役

将部件标记为 retired，并记录原因。退役后系统应拒绝再次安装。

功能六：飞行日志登记

记录飞机飞行任务，并支持统计某一时间段内某部件所经历的飞行次数、累计飞行时长，或所在飞机的使用情况。

功能七：生命周期追溯

给定一个部件编号，系统应能输出其完整生命周期，至少包括：
基本属性；
历史安装记录；
维修记录；
退役情况；
当前状态。

八、建议重点实现的数据库技术

本项目的关键不是页面，而是数据库设计，重点体现以下技术能力。

1. 时间区间建模

安装记录建议使用“开始时间 + 结束时间”方式表示有效区间。结束时间为空表示当前仍在安装中。你们需要思考如何防止同一部件出现多个重叠区间。

2. 唯一性与条件约束

可通过唯一约束、部分唯一索引、检查约束或触发器来实现“同一时刻只能有一条有效安装记录”之类的业务规则。

3. 事务设计

例如“更换部件”不是单条语句，而是一个完整业务过程：关闭旧安装记录、检查新部件可用性、插入新安装记录、更新部件状态。该过程必须放在事务中完成，否则容易出现中间状态不一致。

4. 历史数据保留

系统不应只存“当前状态”，而应能够回答“过去发生了什么”。这是本项目最重要的设计思想之一。

5. 追溯查询

可以通过多表连接、窗口函数、递归查询或视图来实现生命周期追溯与统计分析。

九、建议的事务场景

每组至少完成两个事务场景，建议从以下场景中选择。

场景一：部件更换事务

步骤通常包括：

关闭旧部件当前安装记录；

检查新部件是否可安装；

新增新部件安装记录；

必要时更新相关状态。

任一步骤失败，整个事务回滚。

场景二：退役处理事务

步骤通常包括：

检查部件当前是否仍处于安装状态；

若仍安装，则不允许直接退役，或要求先拆卸；

写入退役记录；

更新部件状态为 `retired`。

场景三：维修完成事务

可包括：

登记维修工单；

更新维修结果；

根据维修结果调整部件状态。

十、必须展示的非非法操作拦截

每个组必须现场演示至少三类非法操作被系统拒绝（不限于以下三种）：

1. 对已退役部件执行安装操作；

2. 为一个已经处于安装状态的部件再次创建有效安装记录；
3. 试图直接删除核心业务数据，或通过覆盖更新破坏历史记录。

十一、AI 使用要求

本项目允许使用 AI 工具辅助开发，但禁止将 AI 输出未经审查直接作为最终答案提交。项目报告需包括 **AI 使用审计报告**，报告至少包含以下内容：

1. 你们在哪些环节使用了 AI；
2. AI 给出的哪些设计或代码是错误的；
3. 你们如何识别这些错误；
4. 你们最终如何修正。

十二、实现建议

建议采用 MySQL/PostgreSQL 作为数据库，应用层可以使用 Flask、FastAPI、Node.js，或命令程序。前端不做复杂要求，不考核炫酷界面，不要求微服务，不要求分布式部署。数据库必须是系统核心，而不是把规则都写在前端里。

十三、提交材料

每组最终需提交以下材料：

1. 项目说明文档
应包含需求分析、ER 设计或逻辑模式设计、关键约束说明、事务说明、业务假设说明。
2. 数据库建库脚本
包括建表、约束、索引、触发器、视图、初始化数据等。
3. 应用代码
可为 Web 应用或 CLI 程序，但必须能演示核心流程。
4. AI 使用审计报告
必须单独成文，不得缺省。
5. 答辩演示材料
生命周期追溯演示、约束拦截演示、事务演示。（录屏）

十四、评分标准

1. 数据模式设计（20 分）

是否能够抽象出合理实体与关系；
表结构是否规范；

是否区分部件型号与部件实例；
是否体现生命周期与历史记录。

2. 完整性约束设计（20 分）

主键、外键、唯一约束、检查约束是否合理；
关键业务规则是否真正落到数据库层；
是否存在明显可被非法写入破坏的数据漏洞。

3. 增删改逻辑正确性（20 分）

安装、更换、维修、退役逻辑是否正确；
是否避免了覆盖历史；
是否采用软删除/退役标记而非随意物理删除。

4. 事务设计（15 分）

是否识别出需要事务保护的业务操作；
事务边界是否合理；
失败时是否能正确回滚。

5. 查询与分析能力（15 分）

是否能实现完整生命周期追溯；
查询是否具有业务解释力；
是否不仅能“查当前”，还能“查历史”。

6. AI 使用与错误反思（10 分）

是否真实记录 AI 参与过程；
是否能指出 AI 输出中的错误；
是否体现出独立审查和修正能力。

十五、加分项

能够区分“当前状态表”和“历史事件表”的设计思想；
能够用数据库机制而不是纯接口逻辑保证规则；
能够给出若干有解释力的统计分析，例如某型号部件平均维修间隔、不同飞机部件更换频率、退役原因分布等；
能够在文档中说明业务假设，而不是默认所有规则都唯一正确；
能够展示对 AI 输出的批判性审计，而不是把错误留到演示时暴露。